

Collaboration and Competition: Solving the Unity Tennis Environment using Multi-Agent Soft Actor-Critic (MASAC)

Aris Markogiannakis

June 29, 2026

Abstract

This report details the implementation and training of a reinforcement learning agent to solve the Unity ML-Agents Tennis environment. The objective is to train two agents to cooperatively keep a ball in play over a net for as long as possible. To achieve this, a Multi-Agent Soft Actor-Critic (MASAC) algorithm was employed, utilizing the paradigm of Centralized Training with Decentralized Execution (CTDE). The environment was successfully solved, achieving an average score of +0.5 over 100 consecutive episodes.

1 Introduction

The Tennis environment consists of two agents controlling rackets to bounce a ball over a net.

- **State Space:** Each agent receives a local observation of 24 variables, representing the position and velocity of the ball and their respective rackets.
- **Action Space:** Each agent outputs a continuous action vector of size 2, corresponding to movement toward/away from the net and jumping. Actions are clamped between $[-1, 1]$.
- **Reward Function:** An agent receives a reward of +0.1 for hitting the ball over the net and -0.01 if the ball hits the ground or goes out of bounds. To maximize the score, agents must learn to collaborate rather than compete.

2 Methodology

To solve this continuous control, multi-agent problem, a Multi-Agent Soft Actor-Critic (MASAC) architecture was implemented.

2.1 Soft Actor-Critic Objective

Standard reinforcement learning seeks to maximize the expected sum of future rewards. Soft Actor-Critic modifies this objective by incorporating an entropy bonus $\mathcal{H}$, which encourages the agent to maximize both reward and action randomness to prevent premature convergence:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))] \quad (1)$$

where α is the temperature parameter governing the exploration-exploitation trade-off.

2.2 Centralized Training with Decentralized Execution

The algorithm utilizes two distinct types of networks:

1. **Decentralized Actors:** Each agent maintains its own policy network $\pi_{\theta_i}(a_i|s_i)$, which maps local observations to actions during inference.
2. **Centralized Critic:** During training, a centralized critic evaluates the combined actions of all agents given the joint state space.

The centralized twin Q-networks (Critics) are updated by minimizing the Mean Squared Error (MSE) against a soft Bellman target:

$$y = r_i + \gamma(1 - d) \left[\min_{j=1,2} Q_{\text{target},j}(\mathbf{s}', \mathbf{a}') - \alpha \log \pi_{\theta_i}(a'_i|s'_i) \right] \quad (2)$$

$$L_Q(\phi) = \mathbb{E}_{\mathcal{D}} [(Q_\phi(\mathbf{s}, \mathbf{a}) - y)^2] \quad (3)$$

The Actor networks are updated by maximizing the expected Q-value from the critic while maintaining high entropy:

$$L_\pi(\theta_i) = \mathbb{E}_{s \sim \mathcal{D}} [\alpha \log \pi_{\theta_i}(a_i|s_i) - Q_\phi(\mathbf{s}, (a_i, \mathbf{a}_{-i}))] \quad (4)$$

3 Implementation Details

Several crucial engineering steps were required to stabilize the training process:

- **Decoupled Credit Assignment:** To prevent agents from being rewarded for the other agent’s mistakes, the reward array was decoupled. The critic evaluates each agent against its specific individual reward rather than a shared team maximum.
- **Auto-Tuning Temperature (α):** The temperature parameter was made dynamically trainable. The minimum clamp was lowered to $e^{-10.0}$ to allow the agents to naturally reduce their entropy and transition into pure exploitation during the late stages of training.
- **Soft Updating:** The target networks were updated using Polyak averaging with a parameter of $\tau = 0.005$ to prevent maximization bias and the moving-target problem.

3.1 Hyperparameters

Hyperparameter	Value
Replay Buffer Size	1,000,000
Batch Size	256
Discount Factor (γ)	0.99
Soft Update Rate (τ)	0.005
Learning Rate (Actor/Critic/Alpha)	3×10^{-4}
Hidden Layers	(256, 256)
Warmup Steps	2,000
Updates per Step	1

Table 1: MASAC Training Hyperparameters

4 Results

The environment was successfully solved (achieving an average score of +0.5 over 100 consecutive episodes) after 2525 episodes. As expected in Multi-Agent environments, the learning curve exhibited a long "warmup" phase where scores remained near 0.0, followed by a rapid exponential increase once a cooperative rally strategy was discovered.

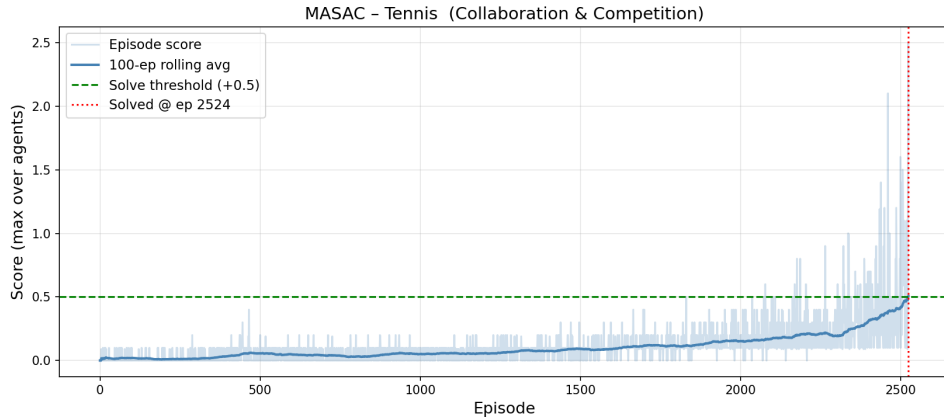


Figure 1: Training curve showing the maximum agent score per episode and the 100-episode moving average. The green dashed line indicates the +0.5 solve threshold.

5 Conclusion and Future Work

The implementation of MASAC successfully modeled the collaborative dynamics required to solve the Tennis environment. Future work could explore tuning the network size to (128,128) to speed up convergence or increasing the `updates_per_step` multiplier to maximize sample efficiency from the replay buffer.